Tecnológico de estudios Superiores de Villa Guerrero.

Ingeniería en Sistemas Computacionales

UNIDAD DE APRENDIZAJE:

SISTEMAS OPERATIVOS

PRÁCTICA:

PRACTICA 5. Rice Contidion.C

PRESENTA:

HERNANDEZ MILLAN LETICIA

GRUPO:

3501

DOCENTE:

ING. REYNOSO ELIAS JUAN

LA FINCA, VILLA GUERRERO, ESTADO DE MÉXICO.

OCTUBRE DE 2026

INTRODUCCION

Las condiciones de carrera (Race Conditions) son situaciones que ocurren en la programación concurrente cuando varios hilos intentan acceder o modificar un mismo recurso compartido simultáneamente. Este problema puede ocasionar errores en los resultados debido a la falta de sincronización entre los procesos. En esta práctica se utiliza el lenguaje C y la biblioteca pthread para crear dos hilos que incrementan una variable compartida llamada contador. Además, se implementa un mecanismo de sincronización denominado mutex, cuya función es controlar el acceso a la variable y garantizar que las operaciones se realicen de manera segura y ordenada.

DESARROLLO

Para realizar la práctica, se desarrolló un programa en C que utiliza la biblioteca pthread.h para trabajar con múltiples hilos de ejecución. Primero, se declaró una variable global llamada contador, inicializada en cero, y se definió un mutex para proteger su modificación.

Posteriormente, se creó la función incrementar (), encargada de aumentar el contador un millón de veces. En la función principal main(), se crearon dos hilos, identificados como h1 y h2, los cuales ejecutan simultáneamente la misma función.

Para evitar condiciones de carrera, se utilizaron las funciones pthread_mutex_lock () y pthread_mutex_unlock (). La primera bloquea el acceso a la sección crítica mientras un hilo modifica la variable, y la segunda libera el acceso para que otro hilo pueda realizar la operación. Finalmente, mediante pthread_join (), el programa espera a que ambos hilos terminen su ejecución antes de mostrar el resultado.

Como cada hilo incrementa el contador un millón de veces, el resultado esperado es de 2,000,000. De esta manera, se comprueba la importancia de utilizar mecanismos de sincronización cuando varios hilos trabajan con una misma variable compartida.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int contador = 0;
pthread_mutex_t lock;  // mutex para sincronizar

void *incrementar(void *arg) {
    for(int i=0; i<1000000; i++) {

       pthread_mutex_lock(&lock);   // bloquear acceso

      contador++;

       pthread_mutex_unlock(&lock); // liberar acceso
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t h1, h2;

    pthread_create(&h1, NULL, incrementar, NULL);
    pthread_create(&h2, NULL, incrementar, NULL);

    pthread_join(h1, NULL);
    pthread_join(h2, NULL);

    printf("Valor final del contador: %d\n", contador);
    return 0;
}
```

```
Leticia@leticia-hplaptop15dy2xxx:~$ nano condition.c
Leticia@leticia-hplaptop15dy2xxx:~$ ./condition.exe
Valor final del contador: 2000000
Leticia@leticia-hplaptop15dy2xxx:~$
```

CONCLUSION

Mediante esta práctica se comprendió la importancia de prevenir las condiciones de carrera en los programas que utilizan múltiples hilos de ejecución. El uso de mutex permitió controlar el acceso a la variable compartida y evitar que ambos hilos modificaran el contador simultáneamente, lo que podría ocasionar la pérdida de incrementos. Asimismo, se identificó que las funciones pthread_create () y pthread_join () permiten administrar la creación y finalización de los hilos. En conclusión, la sincronización es fundamental para desarrollar programas concurrentes confiables, ya que ayuda a mantener la integridad de los datos y garantiza resultados correctos.